

Chapter 1

Getting Started with Visual Basic .NET

This chapter gets you started with Visual Basic 2010. It first explains how a computer works and introduces a general approach to writing a computer program. After a quick review of the development of Visual Basic, the chapter guides you step by step to create your first Visual Basic .NET project. The process includes starting up the Visual Studio Integrated Development Environment (IDE) and initializing IDE settings, creating a new Visual Basic project from a template, and executing a program in the IDE. A large part of the chapter is dedicated to the use of Windows controls, control properties, and events. You will follow examples to learn how to create Windows controls including forms, buttons, and text boxes, as well as how to set up control properties while designing a Windows application user interface and how to use write code in appropriate control events to set property values. Finally, the chapter explains folders and files that make up a Visual Basic 2010 project.

1. Computer and Programming

1.1 Computer Hardware

The hardware is the body of the computer. Core hardware components of a computer include the Central Processing Unit (CPU), the Random Access Memory (RAM), hard drives, and the motherboard. The CPU is the brain of a computer. It handles instructions and processes data. Instructions are usually programs stored on the hard drive on which data are also preserved. A hard drive contains a set of metal disks on which information is stored as magnetism. A new generation of hard drive called solid-state drive (SSD) uses integrated circuits assemblies instead of disks to store data. The CPU can process data at lightning speed. However, hard drives cannot keep up with the speed of the CPU in supplying instructions and data and storing computation results. Therefore, RAM is used to buffer instructions and data and acts like the human's short-term memory. A memory chip is an electronic device that temporarily stores instructions and data for fast access. When you power on a computer, it usually takes a moment for the computer to boot up, during which initial programs (the operating system such as Windows) and data are loaded into the memory for the CPU to process. Unlike the hard drive which is a permanent storage device, the memory stores information temporarily. When the computer is shut off, everything in the memory disappears. Finally, the motherboard of a computer is the chassis on which all devices, including the other three core components and peripheral devices such as keyboard, mouse, monitor, CD drive, etc., are plugged on and gets connected.

1.2 Computer Software

The software is the mind of the computer. The fundamental software to make a computer to "think" is the Operating System (OS) such as Windows, MacOS, Linux, and UNIX. A computer must have an operating system in order to work. Software programs that can be executed in an OS to perform particular tasks, such as ArcMap and Firefox, are called applications. While a

handful of software developers work on operating systems, the majority write programs for applications.

A software program is a list of instructions that tell the computer to carry out tasks. A program must be written in a programming language. There are many different programming languages each having unique semantics and syntax. VB, C++, Java, and Python are currently popular programming languages.

Visual Basic is an intuitive high level programming language because its vocabulary and grammar are more similar to the human language than other languages such as Java, Python, and C++. For example, the following is a few lines of a code for a program that tells the status of a person (Figure 1). You may be able to tell something about the code even if you have no experience in programming.

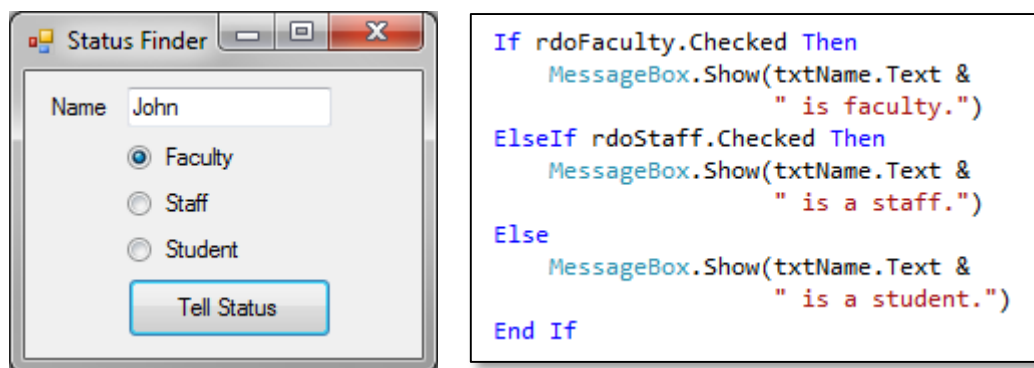


Figure 1 An example of VB code

Although the English-style source code of a program may be straightforward to the human, the text code is not directly understandable to the computer, because the computer hardware can only understand the machine language which is made up of binary codes (0's and 1's). In order for the computer to execute your program, your source code must be converted into the machine language. The process of converting the text source code into binary machine code is called compiling (Figure 2). Each programming language has its own compiling tools.

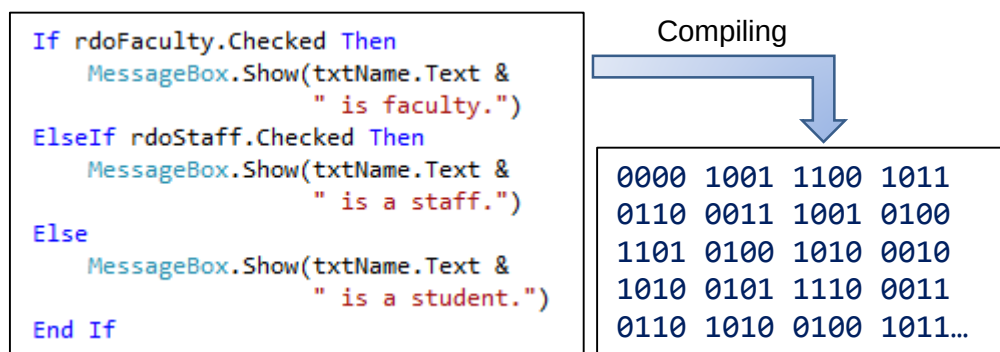


Figure 2 Compiling of computer programs

In Windows, the compiling result of a program is usually an executable file with an extension .exe or a number of dynamic link library (dll) files with the extension .dll. Both .exe

and .dll files are binary files. An executable file in the Windows OS can be executed by double clicking on it in the Windows Explorer.

1.3 How to write programs

Although different programming languages use different semantics and syntax, the basic logics and general procedure to get a job done by programming are common: to progressively break a problem down into small tasks and instruct the computer to handle each task in step-by-step manner. An overall procedure to get a job done often looks like the following:

- 1) Get input (data)
- 2) Process data
- 3) Present results

With the above procedure, each step may need to be further broken down into smaller tasks that can be handled by a programming language.

To write a program is to organize a sequence of actions in a proper order. In fact, we follow orders to do anything. For example, we follow the following steps to make sandwiches:

First, get necessary ingredients: bread, cheese, ham, tomato

Second, make the sandwich

- 1) Pick up a slice of bread
- 2) Add a slice of ham
- 3) Add a slice of cheese
- 4) Add a few slices of tomato
- 5) Add another slice of bread
- 6) Put it in a plate

Third, serve the sandwich

Before we write a program, we often need to figure out a procedure that consists of discrete steps to get the job done. There are two techniques that help us plan the order of actions for a program – flowchart and pseudo code.

Suppose we are to write a program to calculate the average grade of the midterm exam of a class given list of student grades, we may use the following flowchart to plan a procedure for the job (Figure 3).

In a flowchart, input and output are usually denoted by parallelograms, data processing actions are denoted by rectangles, flow directions are indicated by arrowed lines, decisions are denoted by diamonds, and terminals (start/stop) are denoted by capsules.

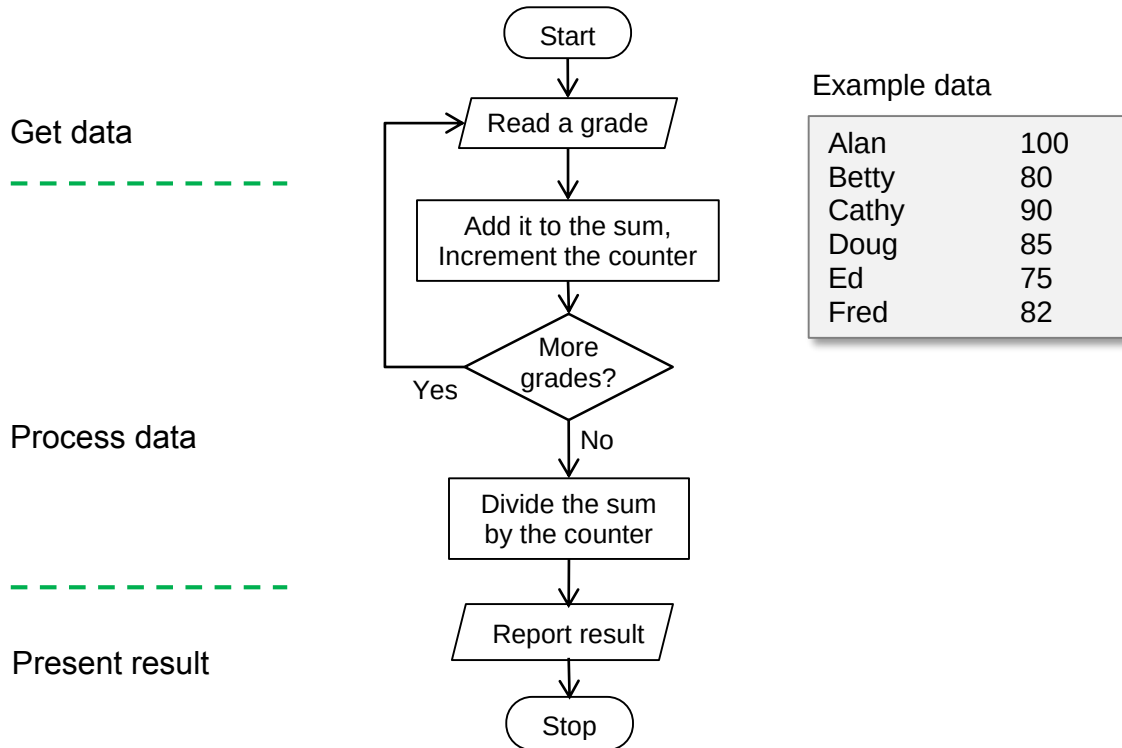


Figure 3 Flowchart for calculating the average score

Pseudo code is an informal description of a computing procedure with some program taste. You may use your own language in the description, but the list of instructions are formatted like a program. The average test score calculation job may be described in the following pseudo code.

```

Declare variable sum
Declare variable counter
Declare variable result
Let sum = 0
Let counter = 0
Loop if a new score is available
    Read a score
    sum = sum + score
    counter = counter + 1
Loop back
result = sum / counter
Display result
  
```

2. Visual Basic .NET

Microsoft introduced Visual Basic (VB) 1.0 in 1991 for Windows. In 1998 Visual Basic 6.0 (VB6) was released as part of Microsoft's Visual Studio (VS) package which includes several other programming languages. VB6 had evolved into a fully Component Object Model (COM) based Integrated Development Environment (IDE), which had gained popularity and been proven a

success. COM defines standards for Windows object interfaces as well as a language-neutral way of object implementation which is separated from interface definition. With well-defined interfaces, Windows components such as buttons and text boxes can be used by developers in applications development without knowing their internal implementation. For example, a button component has an interface including properties such as width, height, border style, and caption. To create a button object, a developer can easily modify the button appearance by setting proper values to these properties. The COM model and user-friendly IDE of VB enable Windows application developers to easily create standard Windows Graphic User Interfaces (GUI).

Microsoft had stopped major development in the COM-based VB after version 6. Because of its popularity, however, VB 6 did not retire until 2012 after serving programmers for 14 years. In 2002, Microsoft introduced a new Visual Basic language based on its .NET Framework as part of the Visual Studio .NET. Like the COM model, .NET also defines objects interfaces, but it has more features such as better language interoperability which allows objects created in one language to be used in certain other languages. Microsoft provides free download and installation of a software environment known as the Common Language Runtime (CLR) that allows programs created with .NET Framework to run on any Windows computer. Most new Windows computers are preinstalled with the latest .NET Framework. Because Visual Basic 2002 and newer versions are based on the .NET Framework, they are often refer to as Visual Basic .NET or VB.NET.

To date, several versions of VB.NET have been released, which include 2002, 2003, 2005, 2008, 2010, 2012, and 2013, each supported by a different version of .NET Framework. VB 2010 is used in this book since it is the currently stable version that supports ArcObjects SDK 10.2. Unless version relevant operations or information is involved, in which VB 2010 will be used, the programming language will be generally referred to as VB or VB.NET in the rest of this book.

3. Starting up VB 2010



To launch VB 2010 is like to launch any windows application. In the Windows start menu click the “Microsoft Visual Studio 2010” shortcut in corresponding program category. If you are the first time to launch Visual Studio, you will need to go through a few steps to set up a VB development environment in Visual Studio. If you see the screen shown as Figure 4, you need to select the “Visual Basic Development Settings” as the default programming language. Once a Visual Studio environment is initialized, you will not need to do it again.

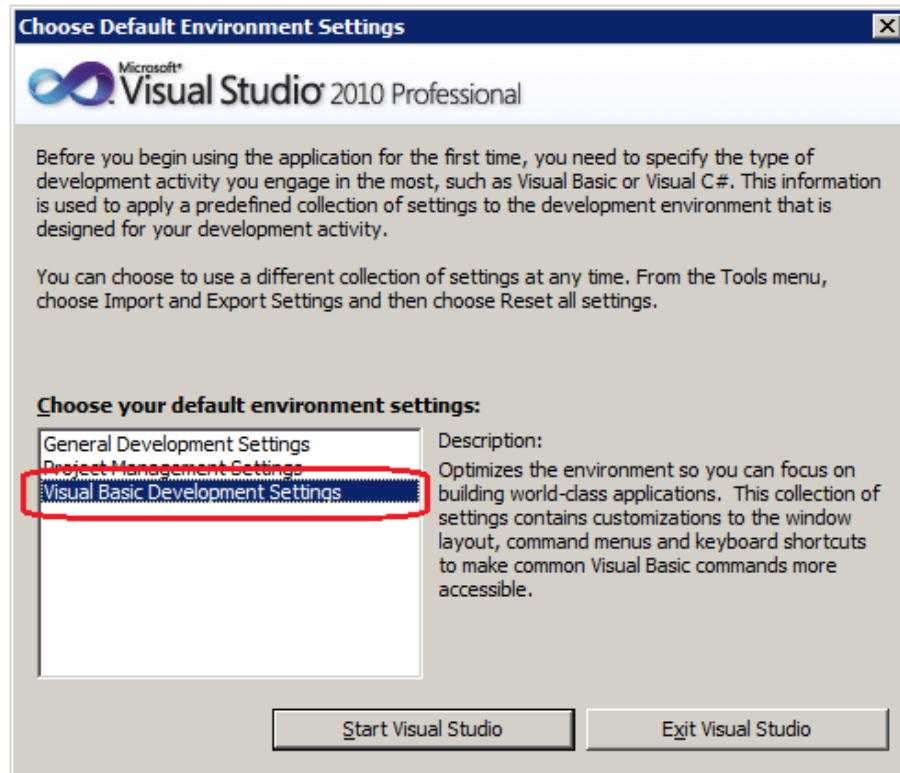


Figure 4 Set the VB development in Visual Studio

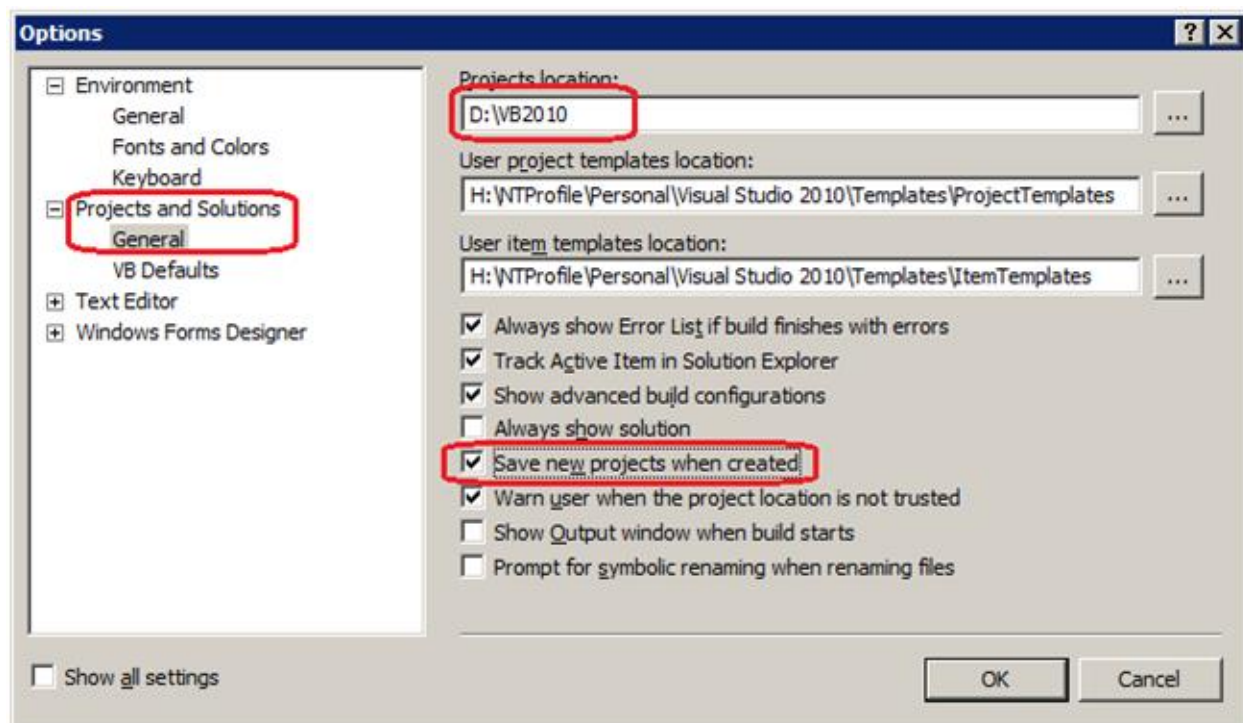


Figure 5 Setting default project location in Visual Studio



The second thing to do when you first launch Visual Studio is to specify a default location for new projects that you will create later. To do so, click the VB 2010 main menu Tools >> Options to open the “Options” dialog box (Figure 5) and select the “General” item under “Projects and Solutions” category in the dialog tree. Then you can specify a default project location in the top box of the dialog. A default project location must be an existing folder that you have writing privilege. Finally, you are suggested to check the “Save new projects when created” checkbox. After setting up a programming environment, you can create a new VB project by clicking on the “New Project” item in the Visual Studio start-up page (Figure 6).

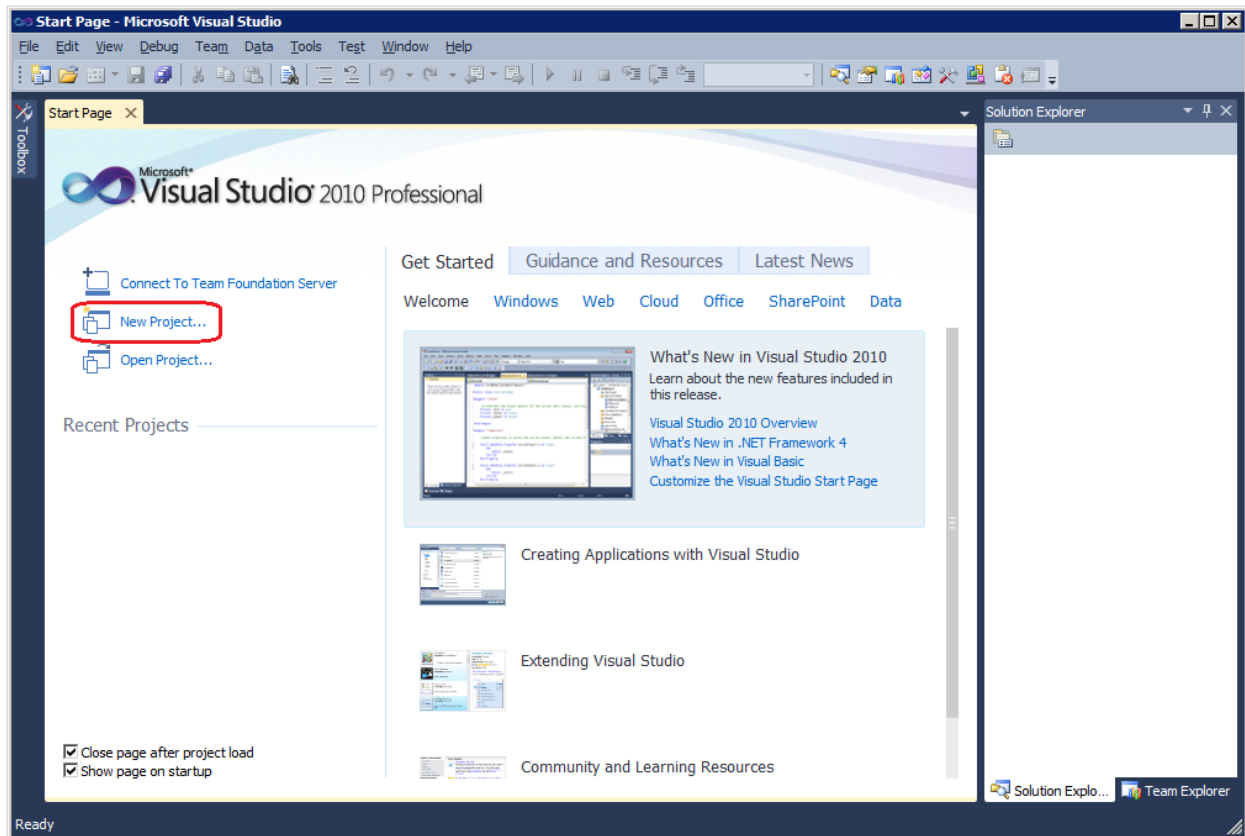


Figure6 Creating a new VS project from scratch



When you create a new project, a New Project template window opens (Figure 7), in which you can select a project type from a list of available templates. In Visual Studio terms, a custom window is called a form. To create a Windows form application, i.e. a program that shows a window when executed, you need to select “Windows” under “Visual Basic” in the “Installed Templates” panel, and then highlight “Windows Forms Application” template in the main panel. You may specify a .NET Framework version for your new project by selecting an appropriate version in the pull-down list in the top of the dialog. To develop ArcGIS 10.2 Add-Ins in VB 2010, .NET Framework 3.5 is recommended. When you create a new project, you often need to provide a project name in the name box in the lower section of the form, and make sure the project is saved in an intended folder. You may often need to keep the “Create directory for solution” checkbox checked. As a testing VB project, please use project name “Test”, and click the “OK” button to finish.

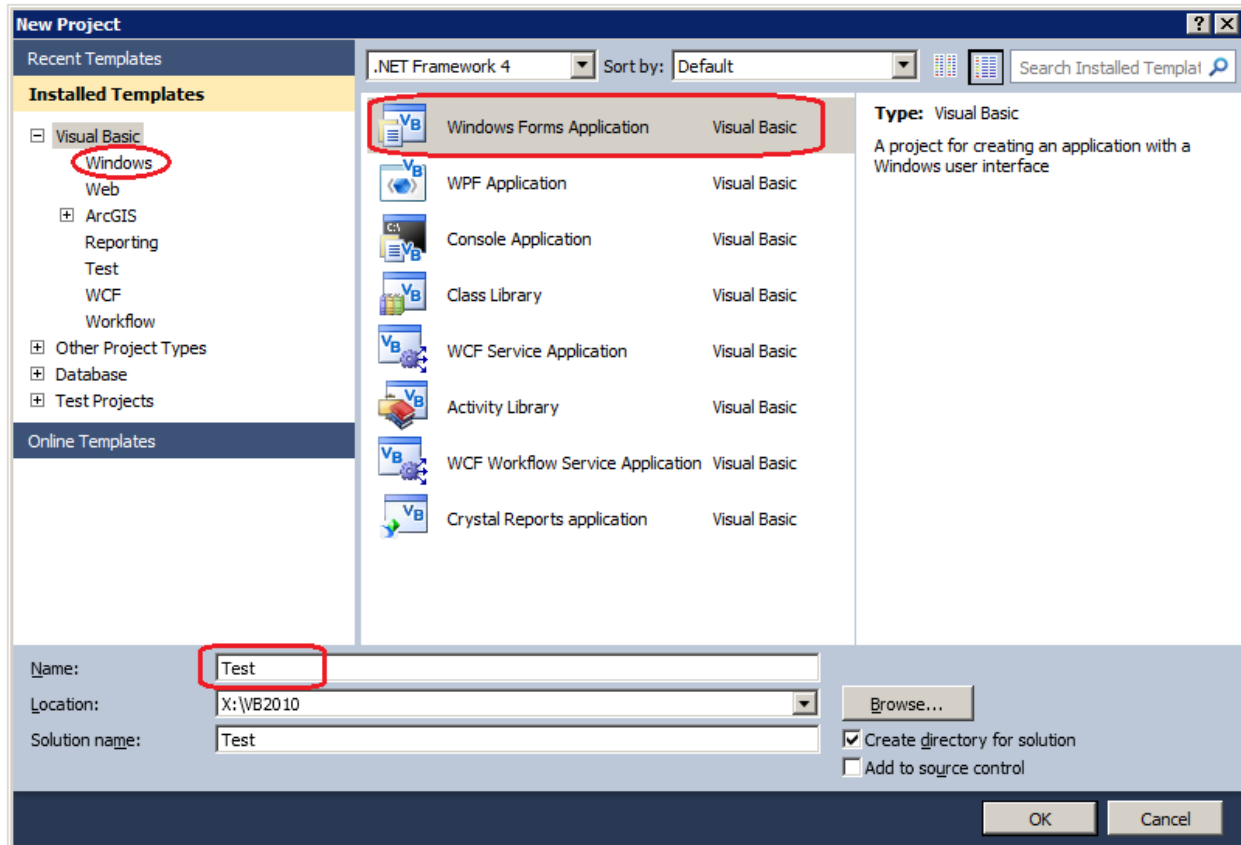


Figure 7 Visual Studio project templates



After a few seconds of preparation, a new Windows form application with a blank form named “Form1” will be created (Figure 8). Congratulations! You have created your first VB project for a Windows form application. Now, you can click the “Start Debugging” button  on the Visual Studio toolbar to run the program. When the program runs a blank window with caption “Form1” displays. You can close the window by clicking the close button on its upper-right corner or clicking the “Stop Debugging” () button on the Visual Studio toolbar. Visual Studio is so wonderful! With a few button clicking, you can create a window without writing a line of code. The same job could require dozens or even hundreds of lines of code in other programming languages.

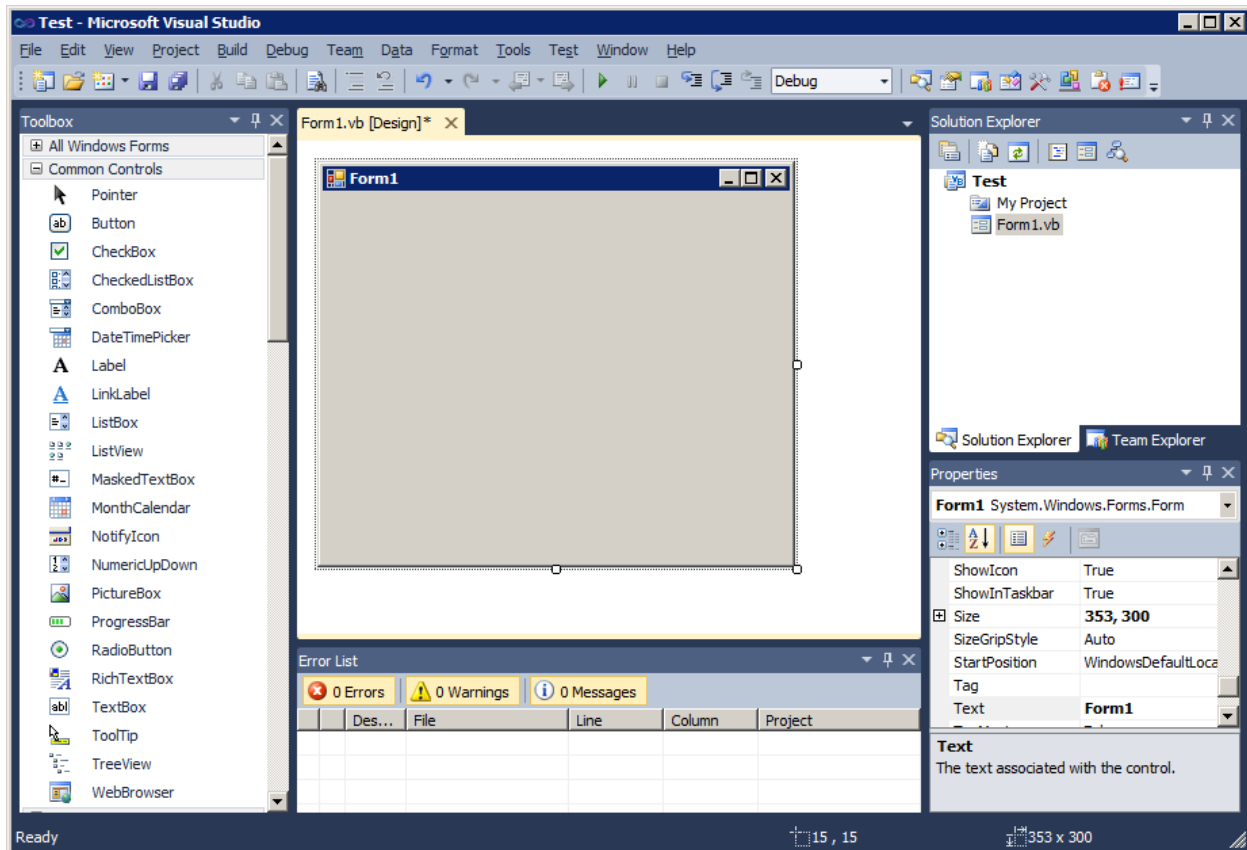


Figure 8 A new VB project for Windows form application

4. Controls, Properties, and Events

A Windows form is a container that holds other Windows controls. In Microsoft terms, buttons, text boxes, radio buttons, check boxes, as well as forms, etc. are called controls. Controls are objects that allow the user to operate or interact. The Windows form you just created does not contain any control yet. This section will show you how to create controls and make them to work for you.

Visual Studio provides templates for all standard Windows controls. To add a control into a Windows form, you just need to select a control template from the Toolbox, which is a panel on the left of the VS window, and draw a rectangle on the form. Please create a couple of controls on the form of your test VB project:



- 1) In the “Common Controls” category in the toolbox, select the “TextBox” tool and draw a rectangle on the form to create a text box control
- 2) Select the “Button” tool from the toolbox and draw a rectangle on the form to create a button control

After creating a control on the form, you can relocate it by dragging the whole control or resize it by dragging its handles, i.e. the little white squares around the control. When you relocate or resize a control, VS lets you to snap and align the edge of the control with other existing

controls on the same form. After adding a few controls into form, you can run the program to test it. To continue to work on the project after testing, you must stop running the program. Normally, when you work on a programming project, you are recommended to frequently save your work. With Visual Studio, every time you run a program, the project is first automatically saved.

4.1 Control properties



VB 2010 is an object-oriented programming language. This means that the language can create objects that have properties and have behaviors (i.e. to perform tasks). As a matter of fact, the form and controls you created in your first project are objects. So they have properties and behaviors.

A property describes an attribute of an object. Properties of a Windows form include width, height, background color, caption text, and so on. Each of these properties describes an attribute of the form object. A text box control has properties such as width, height, text, font type, and font size. You may figure out some properties of other commonly used controls. Different controls have different property sets. But every control must have a property called “Name” which uniquely identifies the control in a project or a form.

Properties values can be set at design time, i.e. the time when you design the form and controls. Except object name, many properties can be set or updated during run time, i.e. the time when a program runs. The Name property can only be set at design time. When you create a new Windows control, a default name is assigned to it by the IDE. For example, a new text box control is assigned a default name “TextBox#” (# is a sequential number), and a new button control is given a name “Button#”. The default name of a new control tells the type of control only but does not reflect the purpose of it. For example, if you create a text box for the user to enter a person’s name and another text box for salary, the default names, such as TextBox1 and TextBox2, by no means tells the different purposes of these boxes. Therefore, it is important to assign a meaningful name to each control you create.



You can assign a name to a control by setting a value to the control’s Name property in the Properties Window (Figure 9). In your test project, please use the Property Window to set a name “frmTest” to the Windows form, a name “txtName” to the text box control, and a name “btnTest” to the button control.

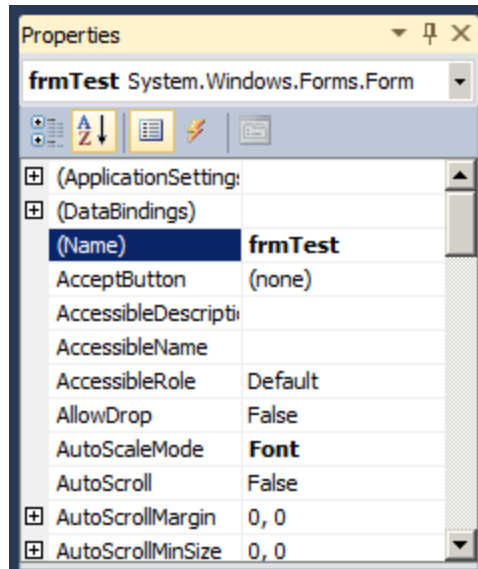


Figure 9 The property window of VB 2010

It is highly recommended that you follow a convention to name controls so that the type and purpose of each control can be easily recognized by its name in a program. In Visual Basic, a conventional style for naming of controls is a lower case type prefix followed by a capitalized word that describes a purpose. It is important to note that a control name must not contain spaces and certain special characters such as slashes, plus and minus signs, and commas and periods. Table 1 shows the conventional type prefixes for some frequently used controls with examples of control names.

Table 1. Naming convention for VB controls

Control type	Prefix	Name Examples
Form	frm	frmMain, frmGIS
Button	btn	btnStart, btnCompute, btnExit
Label	lbl	lblFirstName, lblAge
TextBox	txt	txtLastName, txtSalary
ComboBox	cbo	cboMonth, cboSoilType
ListBox	lst	lstStudents, lstOutput
RadioButton	rdo	rdoMale, rdoFemale
CheckBox	chk	chkSendMail, chkSendEmail

Besides name, you can use the property window to set values to other properties of controls. For example, you can set a precise size (width and height) or preferred background color (BackColor) to the form or the button control. Please use the property window to explore properties of each control in your test project. You will find most of the properties are straightforward and self-explanatory. Then perform the following tasks to your test project:



- 1) Set "My First Windows Application" to the Text property of the form. This changes the caption of the form
- 2) Set "Test" to the Text property of the button control. This changes the face text of the button

- 3) Set a light yellow color to the BackColor property of the text box control.
- 4) Add a label control and name it lblName, then set “Enter a name” to its Text property.
- 5) Resize the form and align the controls as shown in Figure 10.

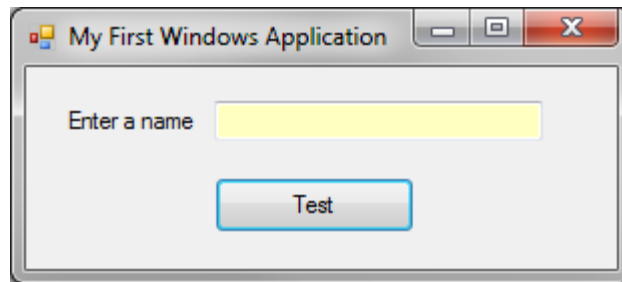


Figure 10 Controls of the test project

Run the project to see result after setting controls properties in your test project.

4.2 Control events

Objects have properties and behaviors. A behavior is an action that an object performs. For example, if you create a dog object, you may want it to bark which is a behavior. In GIS, you may want a map object to have behaviors such as zoom in, zoom out, and pan. In VB programming, forms and controls are objects so they have behaviors. Particularly, a behavior of a form or a control is called an **event**.

An **event** of a control is an action that the control reacts to a user interaction. For example, when you click on a button, a *Click* event is fired; when you push a mouse button down on a form, a *MouseDown* event is fired; when you move the mouse cursor over a form, a *MouseMove* event is fired; when you start to run a Windows program, a *Load* event is fired before the main form displays; when you change the content in a text box, a *TextChanged* event is fired ... You may figure out more events of some common controls.

In VB programming, each event can have an event procedure. An **event procedure** is a block of code that carries out some tasks in response to the event. To see the Click event procedure of the button (btnTest), you just need to double click on the button on form. The Visual Studio IDE will open the code editor and generates a blank event procedure in it. The structure of the mouse clicking event procedure of the btnTest button should look like the following:



```
Private Sub btnTest_Click(sender As System.Object, e As System.EventArgs)
    Handles btnTest.Click
    ' this is where your code will be placed ...
End Sub
```

In the above event procedure, statement `Private Sub btnTest_Click(...)` declares a sub-procedure named “btnTest_Click()”. Words inside the parenthesis (`sender As System.Object, e As System.EventArgs`) define two parameters (*sender* and *e*) that we do not use in most situations. Following the parenthesis, statement “`Handles btnTest.Click`” specifies that this procedure handles the button clicking event. Finally, statement “`End Sub`” defines the end of the procedure. If you are a beginner in VB programming, please be careful with this VB generated procedure structure and avoid making accidental changes to the structure.

VB is an event-based programming language. When you execute a Windows program created by VB, usually nothing happens except a Windows form opens. In order to make the program to carry out a particular task, the user must interact with a control to trigger an event. To make the button clicking event to do something, you can write your code inside the event procedure. For example, you can send a message box to say “Hello, World!” for the button clicking event by adding a line of VB code inside the event procedure as the following.



```
Private Sub btnTest_Click(sender As System.Object, e As System.EventArgs)
    Handles btnTest.Click
    MessageBox.Show("Hello, World!")
End Sub
```

When you start the program and click the Test button, you will see a message box pops up.

A control usually has many events each responding to a different user interaction such as mouse down, mouse move, mouse up, clicking, double-clicking, key down, etc. Different controls have different sets of events. Table 2 shows a few common controls and their frequently used events:

Table 2 Common controls and their frequently used events

Control	Event	Description
Form	Load	When you run a Windows program, this event is fired before the form displays.
	FormClosing	When you close a Windows program, this event is fired while the form is being closed.
	MouseDown	Fired when you press a mouse button on a windows form. This event provides mouse location and button information.
	MouseMove	Fired when you move your mouse on a windows form. It provides mouse location and button information.
	MouseUp	Fired when you release a mouse button on a windows form. It provides mouse location and button information.
	Click	Fired when you click on the form. It is similar to the MouseDown event but it does not carry mouse cursor location and button info.
	DoubleClick	Fired when you double click on the form. No mouse location and button information provided.
TextBox	TextChanged	Fired when the content of a text box changes (such as when the user types in or a program changes the Text property value).
	GotFocus	Fired when the text box gets focus (a cursor blinks in it). A text box gets focus when the user clicks on the control. It can also get focus when the user repeatedly pushes the Tab key to circulate focus among all controls on a form.
	LostFocus	Fired when the text box loses focus. This happens when the user clicks on another control or hit the Tab key to move focus to another control on the same form.
	KeyDown	Fired when the user hits any key on the keyboard while the text box is focused.

Button	Click	Fired when the user clicks on the button. This is the most frequently used event of button controls.
--------	-------	--

Event is an important human-machine interaction mechanism in computer programming. To let a program to perform tasks properly, you must carefully choose proper events of appropriate controls. The most frequently used event of a button control is the *Click* event although a button control also has a *DoubleClick* event and other events. For the form, a very frequently used event is the *Load* event which is fired before the form appears when the user starts a Windows program. We often use the *Load* event to initialize environment parameters for an application. For example, we can use this event to set a precise form size or to set initial values to some variables. When necessary, we can use the *FormClosing* event to perform clean-up jobs, such as closing open files and recycling objects, when we stop a program. For the text box control, the most frequently used event is the *TextChanged* event. This event can be used to carry out certain tasks when you type in a text box. The *KeyDown* is also a very frequently used event of a text box control. It is fired when the user types in the text box. The *KeyDown* event differs from the *TextChanged* event in that this event tells you which key on the keyboard is pushed when it is fired, but the *TextChanged* event does not do so. The *GotFocus* and *LostFocus* events of a control can sometimes be useful. For example, you may want a text box to display a bright background color (such as yellow) when it gets focus and return to normal (white background) when it loses focus.

Each control has a default event which is usually the most frequently used event of the control. For example, the default event of a button control is the *Click* event, the default event of a form is the *Load* event, and the default event of a text box control is the *TextChanged* event. To bring up the default event procedure of a control, you just need to double click on the control in the form design view. If you want to bring up any event procedure, e.g. the *GotFocus* event of a text box control, you can follow this procedure: 1) In the code editor view, select the control in the controls drop-down list; 2) select the event in the events drop-down list (Figure 11). A blank event procedure will be created if it does not exist or scrolled to if otherwise.

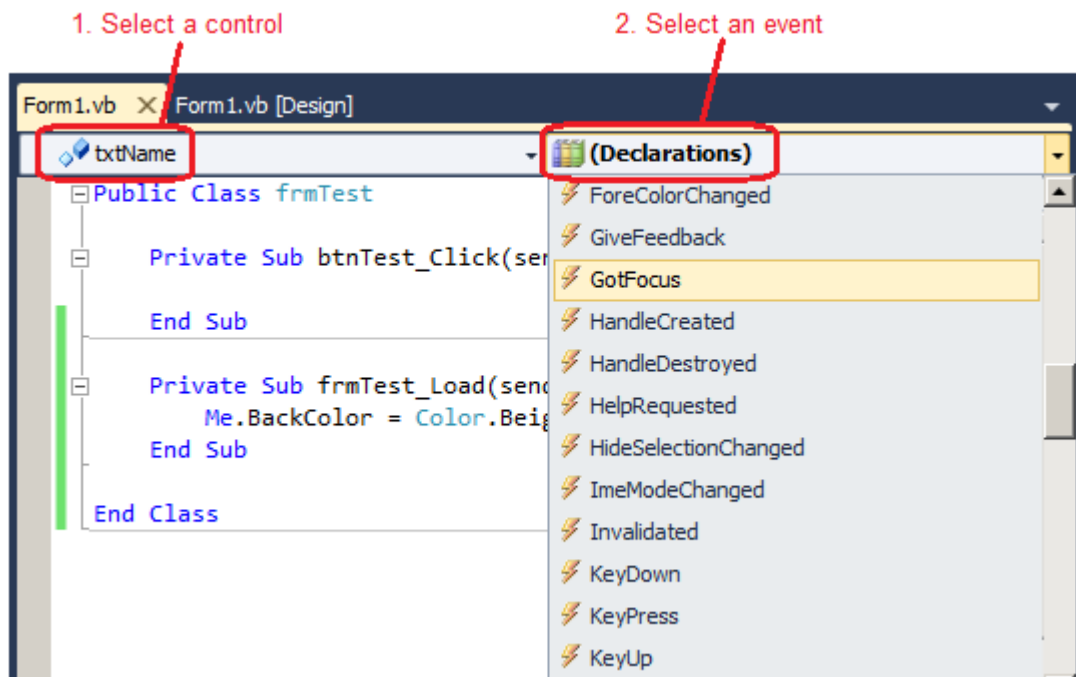


Figure 11 To bring up an event procedure of a control in VB 2010



To practice control events, please bring up the following event procedures:

frmTest: Load, FormClosing, MouseDown, MouseMove, MouseUp

btnTest: Click

txtName: TextChange, GotFocus, LostFocus, KeyDown, KeyUp

4.3 Setting property values by code

In Section 3.1, you learn how to set property values to controls by using the Properties Window during design time. More often, we need to set control property values based on the outcome of certain process while the program is running. For example, we often want to display the result of a calculation in a text box. Therefore, we need to set the calculation result to the Text property of the text box by code. The syntax for setting a value to a control property is:

```
ControlName.Property = value
```



For example,

```
txtName.Text = "Barack Obama"  
txtName.Width = 150  
btnTest.Visible = True
```

In these examples, txtName and btnTest are names of controls. Text, Width, and Visible following a dot symbol are properties of the corresponding controls. Words and numbers that follow equal signs are values being assigned to the properties of controls. Therefore, code line `txtName.Text = "Barack Obama"` is to assign a string value "Barack Obama" to the Text property of text box control txtName, i.e. to display "Barack Obama" in the text box. Similarly, code line `btnStart.Visible = True` is to set a Boolean value True to the Visible property of the btnTest button control, i.e. to make a button visible (although it is commonly not be necessary, but a value False will make a difference).

In programming, string or text values, such as "Barack Obama", must be quoted by double quote signs, but numeric values (e.g. 50) and Boolean values (e.g. True, False) must not be quoted.



Unlike other controls, a form control cannot be referenced by its name in code. For example, it will not work if you try to use the following code to set a caption to the form in the test project:

```
frmTest.Text = "Hello"
```

Instead, you have to use key word "Me" to represent the form. So the following code will work.

```
Me.Text = "Hello"  
Me.Width = 400
```


Since VB is an event-based programming language, if you want to set a value to a control property, your code must be placed in an appropriate event procedure.



Please do the following exercises to practice writing simple code for different event procedures:

- 1) Put the following code in the Load event procedure of frmTest, run the program, and explain what each line of code does.

```
Me.Width = 400  
Me.Height = 300  
Me.BackColor = Color.Beige
```

- 2) Add the following code into the Click event of btnTest, then run the program, enter some text in the text box, and click the Test button. Explain what the code does.

```
MessageBox.Show(txtName.Text)
```

- 3) Add the following code into the GotFocus event of txtName, run the program, enter some text in the text box, and click the Test button. Explain what the code does.

```
txtName.BackColor = Color.Yellow
```

- 4) Add the following code into the LostFocus event of txtName, run the program, enter some text in the text box, and click the Test button. Explain what the code does.

```
txtName.BackColor = Color.White
```



When you write code in the VB code editor, VB continuously provides refined suggestions in a context frame next to where you type. This automatic code completion feature is called *IntelliSense*. IntelliSense is a super smart and extremely helpful feature of the Visual Studio IDE. It relieves you from the burden of remembering all the keywords, variable names, object properties and methods, etc. so that you can focus on getting your job done.

IntelliSense can greatly enhance your learning experience and code writing efficiency. As you keep typing in the code editor, once an intended keyword gets focused, you do not have to finish typing every rest letter of the word. Instead, you can continue to type anything that follows that keyword. IntelliSense can automatically complete the intended keyword. In Figure 12, for example, after you type “me” (IntelliSense will automatically correct case) and a dot, all properties and methods of the form object are listed next to current location. When you type letter “b” after the dot symbol, IntelliSense narrows down to properties and methods that start with letter “b”. Suppose you want to use the BackColor property, as you keep typing, property name “BackColor” will get focused. Then you can continue to type a space without finishing the entire property name. Property “BackColor” will be automatically completed in the code.

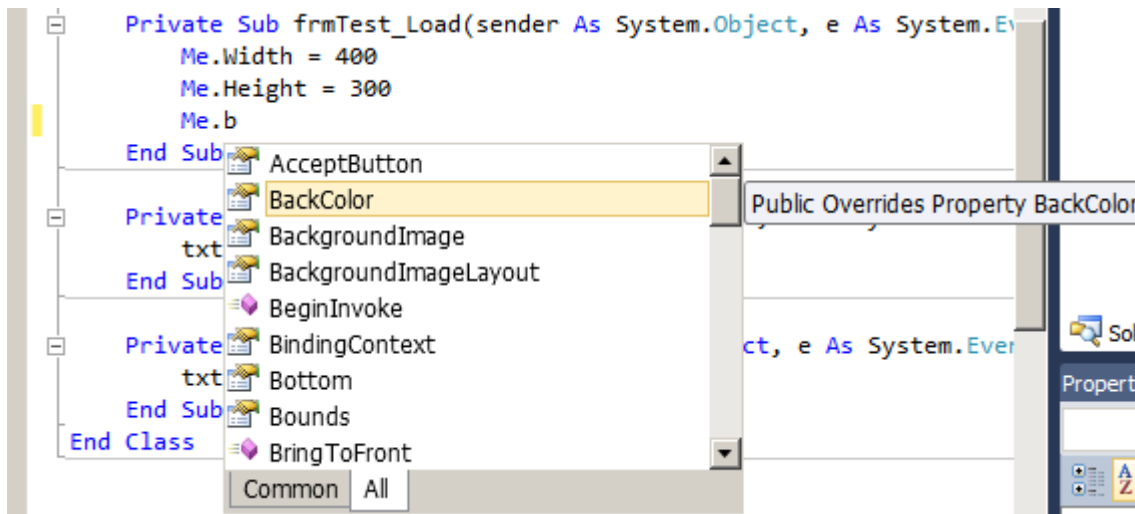


Figure 12 IntelliSense of Visual Basic

With IntelliSense, you do not need to worry about letter cases in VB programming. You may type in lower case in most situations, and let IntelliSense to take care of letter cases.

5. Simple Windows Applications



Example 1: Create a new VB 2010 project named “DataEntry” to facilitate student data entry. When the program runs, it displays a form containing three text boxes, a combo box and a button. The first text box will be used to enter the first name, the second text box for the last name, and the third text box for a student ID. The combo box (a pull-down list) will be used to hold a list of majors (Geography, Planning, Recreation) for the user to select, and the button control, when clicked, will display each piece of student data in a message box. Design the user interface for the project by using specifications in the box of Figure 13.

Control	Name	Property Settings
Form	frmMain	Text: Student Data
TextBox	txtFName	TabIndex: 0
TextBox	txtLName	TabIndex: 1
TextBox	txtID	TabIndex: 2
ComboBox	cboMajor	TabIndex: 3
Button	btnRead	TabIndex: 4
		Text: Read Data
Label	lblFName	Text: First name
Label	lblLName	Text: Last name
Label	lblID	Text: Student ID
Label	lblMajor	Text: Major

Figure 13 Interface design for the data entry project



In VB, each control has a property named `TabIndex`. This property defines the order by which each control gets focus when the user keeps tapping the Tab key on the keyboard. In a VB

project, a control with TabIndex property set to 0 is the first one to get focus when the program is started. By properly setting TabIndex values of controls can enhance data entry efficiency since the user can easily use the Tab key to switch controls during data entry. It is more convenient than to grab the mouse and click on the next control after entering each piece of data.

After finishing the design, run the program to check effect. In the running application window, you can enter text in the text boxes, but there no list item in the combo box for you to select, and clicking the button will cause nothing to happen since you have not added code to respond to the control events.

You could use the Properties Window to add list items in the combo box at design time (by setting the Items property). However, using code to set or update list items in combo boxes is more helpful because in some applications combo box items need to be dynamically updated during run time. Since we want all list items to be available in the combo box as soon as the main form opens, what is the best event to initialize the list items in this project? – The Load event of main form.



To initialize the combo box items for the majors, please double click at any blank space on the form to bring up the Load event procedure. Then add the following code into the procedure.

```
cboMajor.Items.Add("Geography")
cboMajor.Items.Add("Planning")
cboMajor.Items.Add("Recreation")
```

To make the main form to display with a precise initial size, please also add the following code into the form Load event.

```
Me.Width = 263
Me.Height = 212
```

Run the program after adding the above code. You will see the main form appears with the exact specified size and three major names are available to select in the combo box.

To complete the project, please add the following code into the Click event of the button. Finally you can run to test all functions of the project.

```
MessageBox.Show(txtFName.Text)
MessageBox.Show(txtLName.Text)
MessageBox.Show(txtID.Text)
MessageBox.Show(cboMajor.Text)
```

Example 2: Create a new VB 2010 project named “Typewriter” that simultaneously copies text from one text box into another while you type in the box.



First, refer to Figure 14 to design the graphical user interface of the project. Please pay attention to the follow key properties:

- 1) By setting “FixedSingle” to the FormBorderStyle property of the main form, you make the form non-resizable when the program runs.
- 2) To set value True to the Multiline property of both text boxes is to enable multiple lines for these text boxes. When you create a new text box, the default value of the Multiline property is False and the box has a constant height of 20 pixels no matter how you try to resize it. Once you change the Multiline property value to True, you can change the box’s Height property (under the Size property).
- 3) The Size property and Location property are composite properties. You can directly edit the two values separated by a comma or click the little triangle icon to expand the properties to edit their elements (Size property: Width, Height; Location property: X, Y).

Once you finish the design, run the program to check effect.

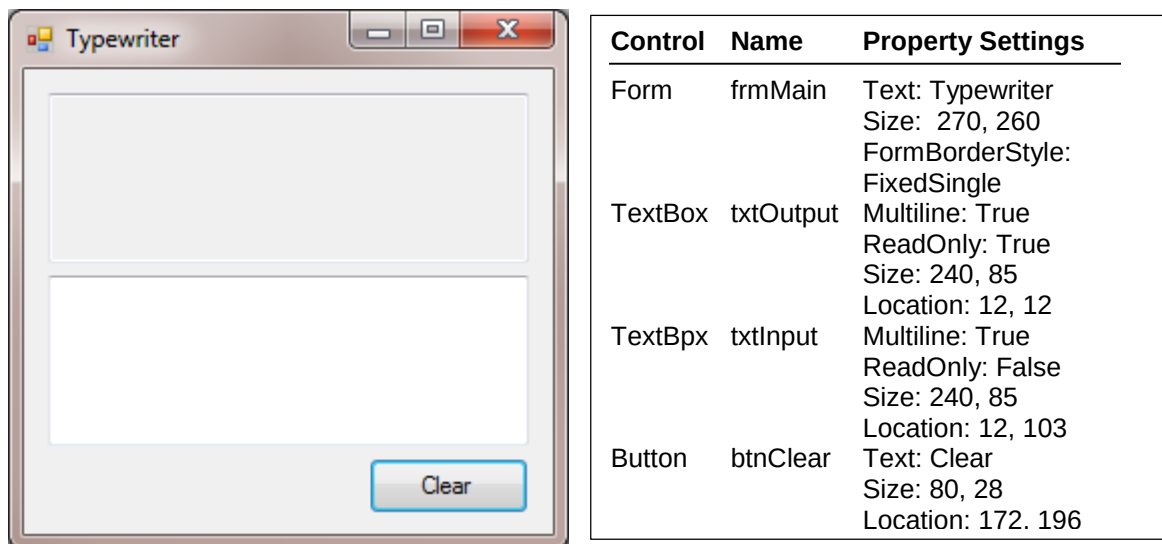


Figure 14 Interface design of the copy text project

If your programming is running, stop it and get ready to write code to implement the text copying function. To open the code editor window, you can click the “View Code” button in the Solution Explorer (Figure 15). If the code editor window is open, you can switch between pages by selecting the page tabs in the main frame or clicking buttons in the Solution Explorer.

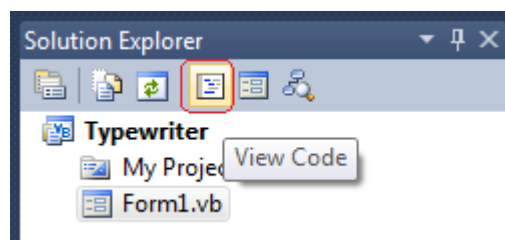


Figure 15 Solution explorer of VB 2010

To copy text from the input text box (txtInput) into the output text box (txtOutput) while you type, you can use the TextChanged event of the input text box. Whenever you type or delete in the input text box the TextChanged event is fired. So that you can write code in this event procedure to carry out text copying. Please bring up the TextChanged event procedure of the input box and write the following line of code inside it:

```
txtOutput.Text = txtInput.Text
```

This line sets the Text value of the input box (txtInput.Text) to the Text property of the output box (txtOutput.Text). In programming, values are assigned from the right-hand side of an equal sign to the left-hand side. Visually, this line of code copies the text in the input text box into the output text box. Now, you can run the program to test its functionality.

To implement the Clear button, bring up the Click event of the button control btnClear and add the following two lines into the event procedure:

```
txtInput.Text = ""
```

This code is straightforward. It simply sets an empty string (a pair of double quotes containing nothing) to the Text property of the box. Once the input box is cleared, the TextChanged event is also fired so the blank content in the box is also copied into the output box, causing the output box to be cleared as well.

6. Understanding Files of a VB.NET project

A Visual Studio solution is a container that contains all items you need for an application. A solution can contain one or more projects. A project is a sub-level container that manages, builds, and debugs the items that make up a specific application. The output of a project is usually an executable program (.exe), a dynamic-link library (.dll), or a module etc. When you start Visual Studio and create a new project from a template, a new project is created in a new solution. When you specify a name to the project, the solution also takes that name by default. In most cases, one solution only contains one project. When you explore a solution in Window Explorer, you will find the following folders and files.

- 1) Under the solution folder, there is a project subfolder and a solution file marked by the extension .sln (Figure 16). The solution file manages all subfolders and files that make the solution. This is the file that you can double click in Windows Explorer to open the VB solution.

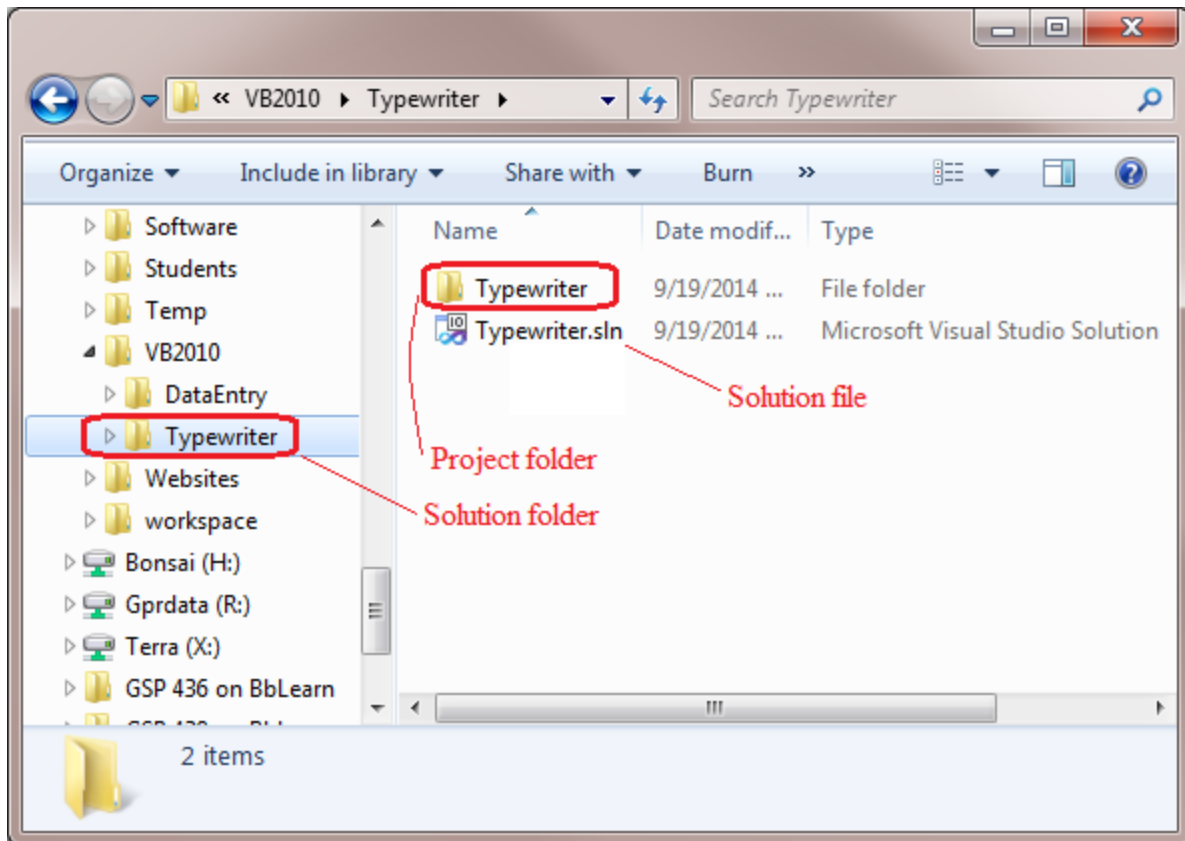


Figure 16 contents of a VB solution

- 2) The project subfolder contains three subfolders, "bin", "obj", and "My Project", and a number of text files (Figure 17). File "Form1.vb" is a text file that contains the code you write to implement control functions. File "Form1.Designer.vb" contain the IDE generated code that defines all the controls you created for the project. The file with the project name and extension .vbproj is an xml file that declares all settings of the project. Please do not delete or modify these files.

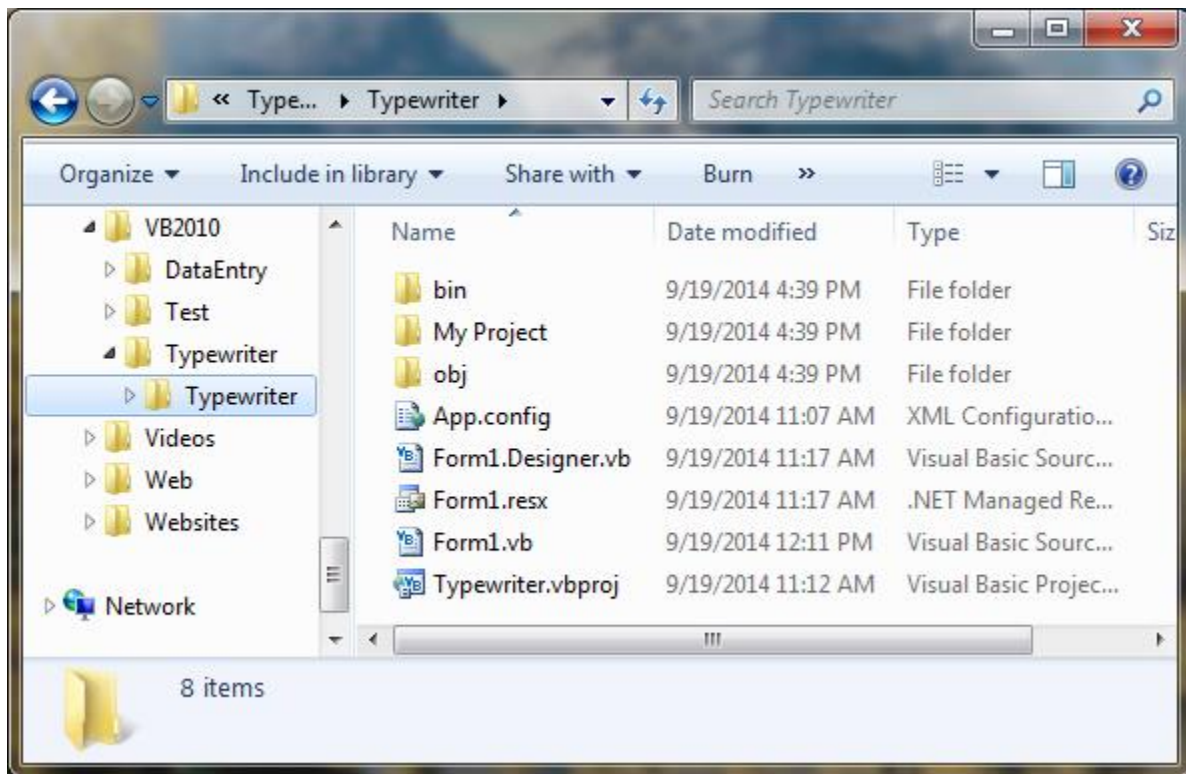


Figure 17 Contents of the project subfolder

The “obj” folder contains intermediate binary files generated in the compiling process. If you delete this folder, it will be created back again when you run or compile the project. Keeping this folder in the project does not hurt. If you drill down the “bin” folder, you will find a number of files under the “Debug” subfolder. The one with extension .exe is the binary executable file that can be executed by double clicking. The executable file is usually the final product of a project. The “My Project” folder holds project data sources, resources, and settings. Please do not change this folder and its contents in windows.

If you need to share a VB solution, you should create a zip archive of the entire solution folder so that all folders and files of the solution are included.

Summary and Highlights

The hardware is the body of a computer. Core hardware components of a computer include the CPU, the RAM (memory), the hard drive, and the motherboard. Programs and data are permanently stored on the hard drive, and are loaded into the memory when the computer is started. The CPU executes programs and processes data derived from the memory and sends results back into the memory. All hardware components of a computer are connected through the motherboard.

The software is the mind of a computer. A software program is a list of instructions that tell the computer to carry out tasks. The fundamental software program that makes a computer to “think” is the operating system. Software programs that carry out particular tasks in an operating system are called applications.

A program written in a programming language is a list of text instructions. The text instructions which are understandable to the human is called source code. Source code must be converted into a computer understandable binary code by a process called compiling.

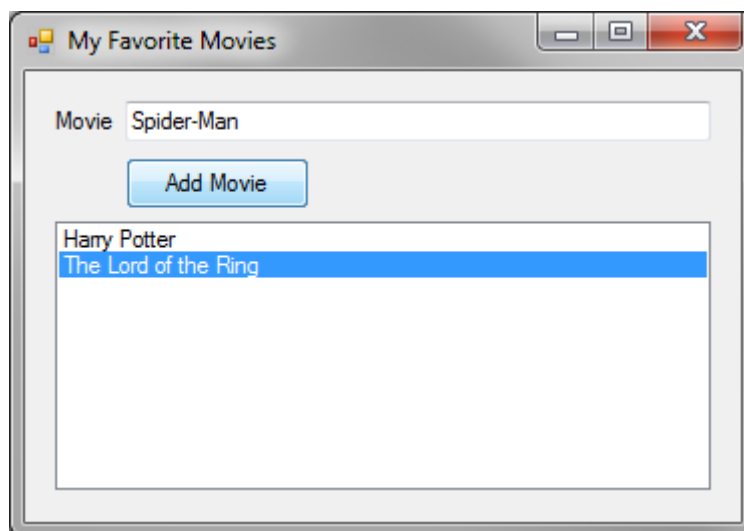
To write a computer program involves progressively breaking a problem down into small tasks and instructing the computer to handle each task in step-by-step manner. To plan a procedure before writing a program, you may need to use flow charts or pseudo code.

VB is a high level programming language. VB 2002 and later versions are called VB.NET because they are based on the Microsoft .NET Framework. Programs created in VB.NET must run in Microsoft .NET Framework run-time environment.

Windows forms, buttons, text boxes, list boxes, etc. are called controls. A control is an object that has properties and events. A property is a piece of data describing an object. Different controls have different sets of properties. Every control must have a Name property. In a VB program, control properties can be set in the property window in design time. Except the Name property, other properties can also be set or updated by code during run time. An event of a control is an action that the control reacts to a user interaction. A control usually has many different events each responding to a different user interaction. The most frequently used event of a button control is the Click event. We often use the Load event of a form control to initialize a program environment or set initial values to certain variables. The action of an event is implemented by custom code in the corresponding event procedure.

Exercises:

1. Answer the following questions.
 - 1) What does each of the core component (CPU, memory, hard drive, and motherboard) of a computer do?
 - 2) Can you name some computer operating systems?
 - 3) Can you name some programming languages?
 - 4) What does compiling do?
 - 5) What is a computer program?
 - 6) What is pseudo code?
 - 7) Why is VB 2002(+) also called VB.NET?
 - 8) What is a Windows form?
 - 9) What is a Windows control? Can you name some frequently used controls?
 - 10) What is a property of a control? Can you name some control properties?
 - 11) Which property of a form object is used to set a form caption?
 - 12) Which property of a form object can make the form not resizable?
 - 13) What are the GotFocus and LostFocus properties of controls?
 - 14) What is the TabIndex property of a control?
 - 15) How do you find all properties of a control?
 - 16) How do you set a value to a property of a control?
 - 17) What is an event? Can you name some control events and situations to use them?
 - 18) Which event of a form object can be used to initialize settings of a program?
 - 19) What is an event procedure?
 - 20) How do you bring up an event procedure of a control?
 - 21) What is IntelliSense? Why is IntelliSense helpful?
 - 22) Can you explain folders and files of a VB 2010 solution?
2. Create a VB project named "Movies". The main form contains a text box, a label, a button, and a list box. When the user enters a movie title in the text box and clicks the button, the movie title is added into the list box and then the text box is cleared. Use the following property settings when you design the graphical user interface.



Note: syntax to add a value into a list box is: `listBox.Items.Add(value)`
where *listBox* is the list box name, and *value* is the text to add

Control	Name	Other Property Settings
Form	frmMovie	Text: My Favorite Movies; Size: 372, 262
TextBox	txtTitle	Location: 49, 15; Size: 292, 20
Label	lblTitle	Location: 11, 18
Button	btnAdd	Location: 49, 43; Size: 92, 26; Text: Add Movie
ListBox	lstMovies	Location: 14, 75; Size: 327, 134

3. Create a VB project named “Books” for book information entry. The main form contains four text boxes, one combo box, a button, and five labels. The text boxes are used for the user to enter book information including book title, author, publisher, and year. When a text box gets focus, the background color of the box turns yellow. When a box loses focus, its background color returns to white. The combo box contains two items for book types, “Non-fiction” and “Fiction” for the user to select. When the button is clicked, each piece of the information entered or selected is displayed in a message box and the text boxes are then cleared. Use the following proper settings when you design the graphical user interface.

Control	Name	Properties Settings
Form	frmBooks	Text: Book Entry
TextBox	txtTitle	TabIndex: 0
TextBox	txtAuthor	TabIndex: 1
TextBox	txtPublisher	TabIndex: 2
Year	txtYear	TabIndex: 3; Text: Enter
ComboBox	cboType	TabIndex: 4
Button	btnEnter	TabIndex: 5